

EMET: An Authority-Incapable Byte-Level Integrity Witness for AI-Assisted Work

Zain Dana Harper

Seattle, WA | [ORCID 0009-0001-7175-5393](https://orcid.org/0009-0001-7175-5393)

<https://github.com/HarperZ9/emet>

July 2026

Abstract

Many recent tools that check the output of AI systems move in one direction. They use richer learned models. Such a model reports a calibrated confidence, sorts errors into several categories, and shows a chain of reasoning. This paper reports the opposite choice, shipped and frozen at version 1.0.0. EMET is a deliberately minimal, model-free, byte-level integrity witness. Byte-level means it works on the exact stored bytes of a file, not on their meaning. Its verdict is a pure function of the recomputed bytes, with no language model in the loop. The central design constraint is not richer output. It is an output type that cannot grant authority: a closed verdict lattice, which is a fixed and closed set of allowed answers, {MATCH, DRIFT, UNVERIFIABLE}. By construction, this set cannot express TRUSTED, APPROVED, or SAFE. We argue that a verifier which cannot grant authority is simpler to reason about against an adversary. There is no model to persuade, and no token that can launder a byte-level match into a permission. We specify the witness as a frozen, normative contract: the requirement keywords of RFC 2119, a pinned output grammar, and a minimal trusted computing base, which is the small set of parts that must be correct for the result to be trusted. We report a conformance harness in which four independent implementations, a Python reference plus Rust, Node.js, and Go ports, agree on 35 core vectors in continuous integration. The Node.js and Go ports were written against the specification and the vectors alone. We are explicit about what this does and does not establish. Same-author agreement demonstrates internal consistency and implementability. It does not demonstrate independent re-derivability, which we name as an open, falsifiable community deliverable. The artifact, specification, conformance vectors, and four implementations are public.

In plain terms

Imagine you give an important document to another person to hold. Later you want to know one thing. Did anyone change it, even by a single letter?

There is a simple way to check. Before you hand it over, you run the document through a small computation that produces a short code. The same document always produces the same code. Change one letter, and the code changes. This short code is called a hash. You write down the code and keep it.

Later you compute the code again from the document you get back. Then you compare it to the code you kept. If the two codes are the same, the document was not changed. If they differ, something was changed. If you cannot read the document at all, you cannot say either way.

Notice what this check can and cannot say. It has exactly three answers. The data matches. The data differs. The check could not be done. It never says the document is good, approved, or safe. Those are different questions, and they are for a person to decide.

This limit is the whole point of the tool this paper describes. The tool is called EMET. Its three answers are named MATCH, DRIFT, and UNVERIFIABLE. MATCH means the data was recomputed and agreed. DRIFT means a real difference was found. UNVERIFIABLE means the check could not be finished. There is no fourth answer. In particular, there is no answer that grants trust or permission.

A checker that can say “approved” or “safe” becomes a target. If an attacker can push it to say “approved” when it should not, the attacker has won. Tools that use a language model to judge can sometimes be talked into the wrong answer. A plain comparison of data cannot be talked into anything. It just recomputes the code and compares. So EMET is built so that the words “trusted”, “approved”, and “safe” are simply not in its vocabulary.

There is one more part. The stored codes live on the machine that made them, and they do not travel. But often you want another person, far away, to check your claim without trusting you and without any shared secret. For this, EMET can write a small file called a receipt. The receipt carries the verdicts and the exact method used to compute them. The other person recomputes everything from the receipt on their own machine. If their result matches the receipt, the receipt is intact. If any part was changed, the recomputation gives a different code and the change is caught.

The last idea is trust in the tool itself. EMET is written so that anyone can rebuild any core verdict from the written specification and the public test cases, without reading the original code. Four working versions of the tool exist, in four programming languages, and they agree on the core test cases. But all four were written by the same author. So they show that the specification is consistent and can be built. They do not yet show that a different, independent author would build the same thing. That last step is left open, on purpose, as a public challenge.

1 Introduction

Plainly: most AI-output checkers keep adding powers, and every added power makes them a better target; EMET removes powers instead, so there is nothing to persuade.

A checker of AI-assisted work faces a steady temptation: to do more. Each new kind of output, a confidence score, a severity label, a suggested fix, makes the checker more useful. It also makes the checker more of an authority, quietly. The danger is built into the structure, not an accident. A checker that can output APPROVED or SAFE is a target for the same pressure that is put on any authority. A checker with a language model inside it can be persuaded, jailbroken, or confused. A plain comparison of bytes cannot.

EMET takes the opposite position. It is an externally-anchored integrity layer. Externally-anchored means the reference it compares against is held outside the system being checked. EMET decides exactly one question: do these bytes re-derive to the same hash as an authorized anchor? An anchor is a stored, authorized fingerprint. EMET refuses every nearby question. Its output type is a closed lattice of three values: MATCH (re-derivation agreed), DRIFT (a confirmed difference), and UNVERIFIABLE (the check could not be completed). The lattice is closed in the formal sense. No implementation may define, emit, or accept any other verdict. In particular, none may emit any value asserting authority or permission. An inability to check is never reported as trust. It is reported as UNVERIFIABLE with a stable machine reason code.

This is the load-bearing distinction we draw against the current trend. Other recent work builds more expressive verifiers: structured reasoning agents with process rewards [1], provenance-native traces, which record where each piece of evidence came from, with per-document influence [2], and governed context vaults [3]. EMET trades expressiveness for a property those systems cannot offer by construction. The verifier cannot be turned into an authority, because the authority words are not in its vocabulary. The contribution is not a better judgment of meaning. EMET is, by design, blind to meaning. It is a witness whose verdict is a pure function of the recomputed bytes, so there is no learned channel through which persuasion

can be applied. We argue this property by construction. We name adversarial evaluation of it as an open deliverable (Section 7).

The paper makes four contributions:

1. **Conceptual:** an authority-incapable witness, formalized as a closed verdict lattice and six testable boundary invariants (Section 3).
2. **Technical:** a minimal-TCB byte-level witness with a portable, content-addressed receipt format that travels off-machine, where anchors cannot go (Sections 3 to 4).
3. **Methodological:** a frozen, normative specification refined by implementations written independently from the reference, where each port surfaced a place the specification was under-determined that the conformance vectors then pinned (Section 5).
4. **Integrity:** an explicit, falsifiable scoping of what same-author agreement proves and what it does not, with the different-author implementation named as an open challenge (Section 7).

2 Background and related work

Plainly: other groups are building richer, learned checkers and provenance layers; EMET is the opposite of these, and it fits underneath them as the small byte-level fact they can call.

Learned and structured verifiers. SEVA [1] trains a verification agent with reinforcement learning. It emits evidence alignments, reasoning chains, a calibrated confidence, and an error diagnosis in six categories. ProvenAI [2] breaks transparency in multi-hop question answering into three parts: answer correctness, citation fidelity, and the influence of each document, measured by removing one resource at a time. Both build richer, learned verifiers. EMET builds a poorer, deterministic one, on the principle that a verifier with no learned component has no persuasive surface.

Provenance and context governance. ContextNest [3] formalizes context governance, meaning provenance, version identity, integrity, and point-in-time reconstruction, as a layer beneath retrieval-augmented generation. Retrieval-augmented generation is a method where a model answers using documents fetched at query time. Minos [4] performs LLM-driven provenance backward tracking for forensic analysis. These systems are natural consumers of a leaf witness. EMET can supply the byte-level integrity fact that a governance stack invokes, without competing with the governance layer itself.

Constrained verifiable agents. *Making Failure Safe* [5] shifts agent output from free-form code to typed, rule-checked configurations. EMET shares the same instinct, a shift toward the checkable, but applies it at the integrity layer rather than at the code-generation layer.

Foundations. EMET rests on content addressing, on Merkle and tamper-evident logging, on capability-based security (the witness holds no key and takes no action), and on the long line of work on reproducible builds. Its departure is not cryptographic novelty. It is the deliberate removal of the authority channel from the output type.

3 Design

Plainly: the design puts the “no authority” rule inside the output type itself, hashes the exact bytes, reports “cannot check” instead of guessing, and states six rules that a test can each break.

3.1 The closed verdict lattice

Every integrity judgment EMET emits is exactly one of {MATCH, DRIFT, UNVERIFIABLE}. This enumeration is closed. An implementation **must not** emit TRUSTED, APPROVED, SAFE, or any value asserting authority or permission. Absence of drift is reported as MATCH (re-derivation

agreed) or as UNVERIFIABLE (no raw-byte anchor), never as trust. This is the first of six boundaries, facts, not authority, and it is encoded directly in the output type.

Auxiliary judgments are closed in the same way, and none of them maps to an authority word. Coherence, the check that a presented view matches its source, emits {COHERENT, VIEW__DIFFERS__FROM}. Corroboration, the check that separate reads of the same target agree, emits {CORROBORATED, QUARANTINE__INTACT}. A marker scan, which looks for known signatures such as injection patterns, emits a count that is a non-negative integer.

3.2 Hashing and identity

The identity of an artifact, meaning a file or piece of work, is SHA-256 over its exact raw bytes. SHA-256 is a standard hash function: it turns any data into a fixed-length code, and any change to the data changes the code. An implementation **must not** normalize, transcode, or canonicalize the data before hashing. Line endings, encoding, and whitespace are part of the artifact. Detecting a CRLF rewrite, that is, a change in the invisible end-of-line characters, is a feature. Bytes are read through a raw channel (`read_binary`), never through a mediated view. With no raw channel, the result is UNVERIFIABLE.

3.3 UNVERIFIABLE, never TRUSTED

If an implementation cannot obtain raw bytes, cannot find an anchor, or cannot complete a check, it **must** report UNVERIFIABLE with a stable machine reason code, not prose. It **must not** substitute a default, a cached value, or a trust assertion. The governed codes are closed: E_NOT_FOUND, E_NO_RAW_CHANNEL, E_NO_ANCHOR, E_NO_CORPUS, E_NO_CORPUS_VERSION, E_NO_SECOND_READ_PATH, E_LOG_CORRUPT. An inability to check is never trust, and it is never a difference.

3.4 Exit codes as the verdict class

The exit code is the verdict class as an integer. 0 means the property held: all MATCH, COHERENT, CORROBORATED, INTACT. 1 means a negative finding: DRIFT or divergence. 2 means the check could not be done: UNVERIFIABLE. 3 means markers were detected. 64 means a usage error. For multiple targets, a confirmed difference dominates an inability to check, so exit 1 dominates exit 2. The exit code is data plus an advisory signal. It allows nothing and denies nothing. Enforcement is a downstream decision, taken on infrastructure the owner controls.

3.5 The six boundaries as testable invariants

The design thesis is encoded as six invariants. Each one is falsifiable by a test:

1. **Facts, not authority.** The lattice is closed. No codepath emits outside it or produces TRUSTED.
2. **Attests, never adjudicates a model-safety decision.** EMET operates only on artifact, byte, and provenance facts. No command takes a model-safety or content decision as input.
3. **Outside, never inside.** Targets are read by raw bytes. EMET must not be hosted by, or routed through, the system it audits.
4. **Advisory by default.** A verdict is data plus an exit code. On its own, EMET allows, denies, blocks, and enforces nothing.
5. **Re-derivable.** Every verdict is reproducible from the same specification version, corpus version, and bytes. There is no secret and no held key.

6. **Zero actuation on the audited target.** EMET writes only to its own implementation-private stores: the anchor store, the hash-chained log, and the redacted `<file>.refused` copy. The operator is the single actuator over the audited world.

3.6 Trusted computing base

The core depends only on the language runtime and its standard library (CPython plus `hashlib`, `json`, `os`, and `subprocess`). It adds no third-party runtime dependency. Optional adapters, for signing or for SARIF or in-toto emission, live in separate packages. The minimal-TCB guarantee applies to the named core only.

3.7 Tamper-evident audit chain

The audit log is JSON-lines. Each entry carries `kind`, `fact`, `prev`, and `chain`, where

$$\text{chain} = \text{SHA256}(\text{prev} + \text{kind} + \text{canonical_json}(\text{fact})),$$

`prev` is the prior entry's chain value, and the genesis `prev` is 64 zeros. Binding `kind` into the chain means that relabeling an entry's operation, for example changing `anchor` to `refuse`, is itself tamper. The `audit` command recomputes the chain and also checks linkage. Each stored `prev` must equal the prior entry's chain, so a forged suffix that is internally consistent and re-chained is caught, not only a single edited entry.

Canonical JSON across languages. `canonical_json` is the serialization with keys sorted, with `,` `"` and `:` separators, and with `ensure_ascii` escaping, UTF-8-encoded before concatenation. A language's default encoder does not produce this form. Go's `encoding/json` HTML-escapes `<` `>` `&`, and neither Go nor JavaScript's `JSON.stringify` matches the spacing or sorts keys. A conforming implementation must therefore hand-roll or post-process its serializer. Pinning this byte form is what lets an auditor re-derive a chain that another implementation wrote.

4 The portable witness receipt

Plainly: the stored anchors stay on one machine, so EMET can also write a small self-checking file that a stranger recomputes on their own machine, with no shared trust and no network.

The anchor store is implementation-private. A raw anchor or a `VERIFY` verdict cannot leave the machine that produced it. The portable witness receipt (`emet-witness-receipt/v1`) is the standardized, cross-implementable form that can travel. With it, a different party re-derives and checks the result on their own machine, with zero shared state, zero trust in the producer, and zero network access.

A receipt is a JSON object. It carries one or more EMET verdicts plus the re-derivation method. Its content address is `receipt_id = SHA256` over the canonical JSON of the receipt with the `receipt_id`, `signature`, and `witness` fields excluded. The witness block, which holds the implementation name and that implementation's own artifact-of-record hash, is excluded from the address because it is producer identity and is intrinsically per-implementation. Including it would make the same subject, verdict, spec, and timestamp hash to different identifiers across implementations, which would defeat the guarantee. The guarantee is this: the same subject bytes, specification version, corpus version, and issuance timestamp yield a byte-identical `receipt_id` across conforming implementations. Tampering any addressed field re-hashes to a different identifier.

The offline verifier `emet check` is stateless. It uses no anchor store, no log, no network, and no shared key. It re-derives `receipt_id` and compares it (a mismatch gives `RECEIPT_TAMPERED`).

It can also re-hash live subject bytes (`--recompute-from-paths`). It returns a member of the closed receipt lattice `{RECEIPT_VALID, RECEIPT_TAMPERED, RECEIPT_UNVERIFIABLE}`, with `TAMPERED` dominating `UNVERIFIABLE` as in Section 3.4. A receipt asserts a fact of re-derivation, never authority. `RECEIPT_VALID` means the receipt’s content address is intact at check time. It maps to no authority word.

5 Multi-language conformance

Plainly: conformance means passing every fixed test case at a stated version; writing the tool again from the spec alone exposed gaps, and the test cases now pin them.

Conformance is defined by extension. An implementation conforms at a given specification version if and only if it satisfies every normative **must** and produces the expected verdict and exit code for every vector in `conformance/vectors.json` at the stated corpus version. A vector here is a fixed test case with a known expected answer.

Four implementations pass the core vectors in continuous integration: the Python reference, plus Rust, Node.js, and Go ports. The Node.js and Go ports were written against the specification and the vectors alone, not the reference code. Each port surfaced specification under-determination that the vectors then pinned. The clearest example is the marker count. The specification initially left `count` implicit, and an independent reimplementer forced the rule, a non-overlapping leftmost scan in corpus order, which the vectors `refuse-three-markers` and `refuse-repeated-marker-occurrence-count` now pin jointly. This is the methodological contribution. The act of independent implementation refined the specification, and the vectors are the artifact of that refinement.

The byte-hash core (anchor, verify, coherence, corroborate, audit) depends only on SHA-256 and exact bytes. It uses no corpus, no key, and no clock. Marker-dependent output (refuse, and the marker census) is versioned data. The corpus is the versioned data the marker rules depend on. Two implementations agree on a marker verdict only when they are pinned to the same corpus version, which each marker-dependent output echoes.

5.1 What the corpus covers

Capability	Vectors	Languages that pass
Core (byte-hash, exit split, JSON envelope, marker path, audit chain)	35	Python, Rust, Node.js, Go
Portable witness receipt (<code>emet check</code>)	5	Python, Rust, Node.js (Go: pending)
Stripped-credential rebind (experimental)	4	Python (cross-language: follow-on)
Total	44	

6 Threat model and honest limits

Plainly: we assume an attacker on the path between the model and the truth, we name the one attack class that would turn the witness into an authority, and we list plainly what the design does not fix.

We model an adversary who controls some part of the path between a model, or its operators, and ground truth. This adversary can alter artifacts, substitute a presented view for the source, inject false authority into context, tamper one read channel, or edit an accountability log. The adversary does not control the operator, the external verifier of record, or the mathematics of SHA-256. Under STRIDE, a standard list of threat categories, the load-bearing class is elevation of privilege. An adversary who could make EMET assert authority, adjudicate a model decision, or run inside the audited system would convert the witness into an authority. The six boundaries are designed to preclude this by construction. EMET’s output vocabulary

contains no authority token, and its verdict is a pure function of the recomputed bytes. This is established by construction, rather than by adversarial testing, which we name as an open deliverable. We also distinguish this from a separate concern the design does not resolve. A downstream integrator remains free to treat a `MATCH` as a permission. So the property established here is that EMET cannot emit authority. It is not that a consumer cannot misuse an integrity fact as one.

We state the residual surface plainly:

- **Denylist incompleteness.** Marker detection is a known-signature denylist. The absence of a marker is not the absence of injection.
- **Trust-root regress.** A compromised execution substrate re-derives a compromised self-hash consistently. `selftest` proves integrity only relative to an uncompromised substrate. An external verifier must be the check of record for EMET itself. EMET must not be its own root of trust.
- **Single-read-path collusion.** Corroboration detects divergence across disjoint reads. An adversary who tampers every read path identically is not caught at the byte layer.
- **Semantic blindness.** EMET judges bytes and provenance, never meaning. A semantically harmful artifact with intact bytes passes `VERIFY` by design.
- **Coerced operator.** The single actuator is assumed uncoerced. Coercion is out of model.

7 Discussion: what agreement proves

Plainly: four versions by one author show the spec is consistent and buildable, not that a stranger would rebuild it; that stranger’s version is the open challenge.

The four implementations share an author. We are therefore explicit about the epistemic status of the conformance result. Same-author agreement across four languages demonstrates two things. It demonstrates internal consistency: the specification does not contradict itself across its surfaces. It demonstrates implementability: the specification can be realized in four runtimes from its text. It does not demonstrate independent re-derivability. That is a stronger claim: that a different author, reading only the specification and the vectors, can produce an implementation that passes them. That implementation is an open, named deliverable. No party should treat re-derivability as proven until it exists.

This distinction is the spine of the artifact’s integrity. A frozen contract plus production-grade reference implementations is a legitimate 1.0 deliverable. Claiming the external re-derivability that only a different author can confer would be exactly the in-band overclaim that EMET exists to strip from other systems. We therefore frame the different-author implementation as a falsifiable community challenge. A single implementation from `SPEC.md` alone, in any language, passing `conformance/vectors.json`, converts re-derivability from an assertion into a demonstrated fact. The artifact is structured to make that challenge cheap to attempt and hard to game: a frozen specification, public vectors, a from-spec check command, and four public implementations to consult only after a genuine independent attempt.

EMET is a leaf witness, not a governance system. A governance stack that decides which artifacts are approved, current, and attributable [3] can invoke EMET for the byte-level integrity fact and retain the authority decision at its own layer. The two concerns are separable by design.

8 Reproducibility

Plainly: the spec, the test cases, the four builds, and the runner are public, so anyone can recompute a core verdict without reading the reference code.

The frozen specification (`SPEC.md v1.0.0`, normative, RFC 2119), the conformance vectors, the four implementations (Python, Rust, Node.js, Go), and the from-spec conformance runner (`conformance/run.py`) are public under MPL-2.0 at <https://github.com/HarperZ9/emet>. A reader can re-derive any core verdict from the specification and the vectors alone. A reader can check any implementation against the corpus without reading reference code.

Availability

EMET, its specification, conformance vectors, and the four implementations are at github.com/HarperZ9/emet (MPL-2.0).

References

- [1] Yuan, A., Nian, Y., Zhang, H., Su, Z., and Zhao, Y. *SEVA: Self-Evolving Verification Agent with Process Reward for Fact Attribution*. arXiv:2606.29713, 2026.
- [2] Faizan, M. and Alharthi, D. *ProvenAI: Provenance-Native Traces of Evidence in Generated Answers*. arXiv:2606.26449, 2026.
- [3] Sulpovar, M., Konsynski, B. R., Kanchwala, Q., and Goodhart, G. *ContextNest: Verifiable Context Governance for Autonomous AI Agents*. arXiv:2607.02116, 2026.
- [4] Wang, J., Li, Z., Wang, Z., Shen, X., and Zhang, F. *Minos: A Multi-Agent Collaborative Framework for Provenance-Based Backward Tracking*. arXiv:2607.00440, 2026.
- [5] Chen, B. *Making Failure Safe: A Constrained, Verifiable Agent Framework for Open-Web Data Collection*. arXiv:2607.00035, 2026.
- [6] Bradner, S. *Key words for use in RFCs to indicate requirement levels*. RFC 2119, IETF, 1997.
- [7] NIST. *Secure Hash Standard (SHS)*. FIPS PUB 180-4, 2015.
- [8] Reproducible Builds Project. *Reproducible Builds: a set of software development practices*. <https://reproducible-builds.org>, 2026.